

Cos’è: Node non è né un framework né una libreria, ma è un runtime che permette l’esecuzione di JavaScript al di fuori del contesto del browser

Come lo useremo: utilizziamo node per il lato server (back-end), tuttavia è impreciso dire che il server è implementato tramite node, node infatti è solo il contesto in cui il server esegue, ma la vera libreria che lo implementa si chiama express.

Installazione di Node

Download dal sito ufficiale: visitate https://nodejs.org/en/download per scaricare il file .pkg. Il sito dovrebbe già impostare il download adeguato per il vostro sistema operativo.

Verifica dell'installazione: una volta installato eseguire i seguenti comandi da terminale (non serve aprire il terminale in una cartella specifica):

```
node -v
```

```
npm -v
```

Se vengono stampati numeri di versione per entrambi i comandi allora l’istallazione è avvenuta con successo.

Node e NPM

La precedente installazione avrà installato due pacchetti:

- Node: node stesso, il runtime di cui parlavo.
- NPM: sta per Node Package Manager, aggiorna, gestisce e scarica librerie per node.js. Per ogni progetto il programmatore ha bisogno di librerie e quindi NPM serve per prendere queste librerie e caricarle nel progetto.

Creare un nuovo progetto in Node.js

Setup iniziale: Per creare un nuovo progetto creiamo una cartella vuota e apriamo il terminale in quella cartella, poi digitiamo su terminale:

```
npm init -y
```

Il file package.json: Verrà creato un file chiamato *package.json*. Questo è un file di configurazione che serva a dare istruzioni a npm. Esso contiene

- Metadati: nome, autore, versione, descrizione

- Comandi di esecuzione: main, scripts (in particolare scripts consente di definire comandi eseguibili da terminale tramite il comando "npm run <nome comando>"

- Dipendenze: ovvero le librerie che il progetto necessita per l'esecuzione

Installazione di Express nel progetto: Express è la libreria che consente l'implementazione del server. Per installarla è necessario aprire il terminale nella cartella del progetto e digitare:

```
npm install express
```

Due elementi sono stati aggiunti:

- un file *package-lock.json*: è sempre un file di configurazione, ma è molto più specifico sullo stato attuale delle dipendenze.

- Una cartella *node modules*: qui ci sono i file sorgente delle librerie che abbiamo scaricato e che servono al progetto.

ATTENZIONE: a differenza di package.json, il file package-lock.json e la cartella node modules NON devono essere assolutamente modificari.

Eseguire un progetto già esistente su node.js

Per questi passaggi è necessario che node e npm siano installati come specificato nelle slide precedenti. Prendete la cartella di progetto e aprite una finestra del terminale al suo interno (nella stessa cartella dove si trova il file *package.json* del progetto) e digitate:

```
npm install
```

In questo modo npm installerà in automatico tutte le librerie necessarie al progetto.

Un esempio: Sip ‘n Snap

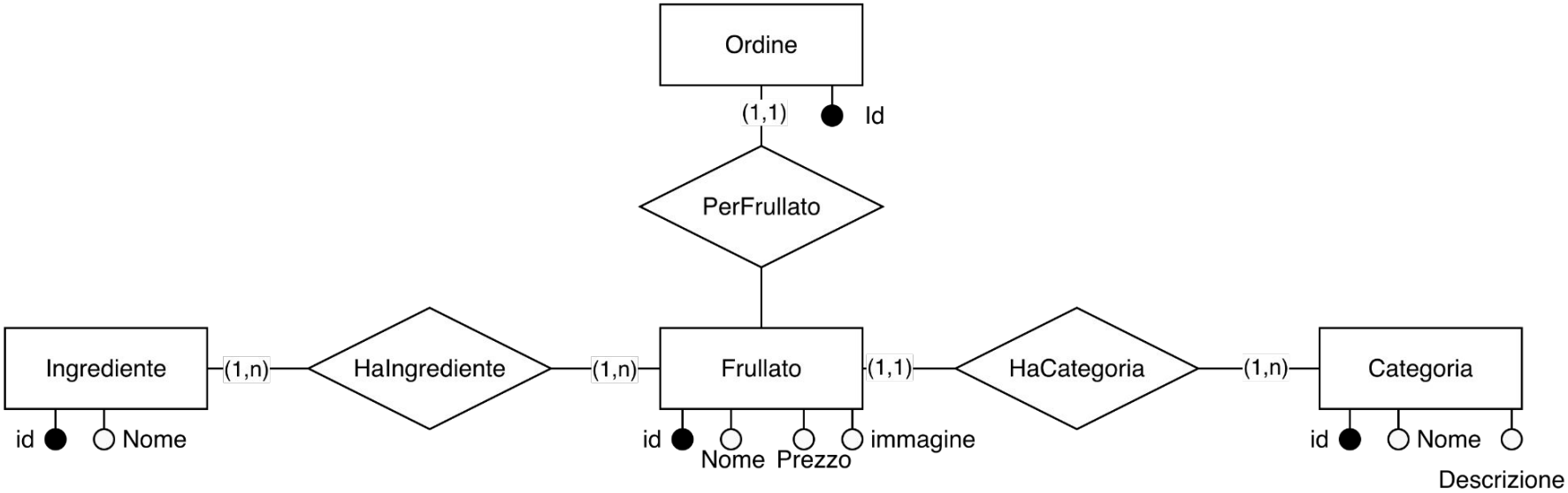
Sip ‘n Snap è un bar di frullati fittizio, il bar ha bisogno di un’applicazione web per gestire le ordinazioni. Tale applicazione utilizza node.js per implementare l’architettura client server.

Sip ‘n Snap ha 4 pagine:

- La home.
- Una pagina per visualizzare i frullati ed effettuare le ordinazioni.
- Una pagina ‘Chi Siamo’
- Una pagina contatti

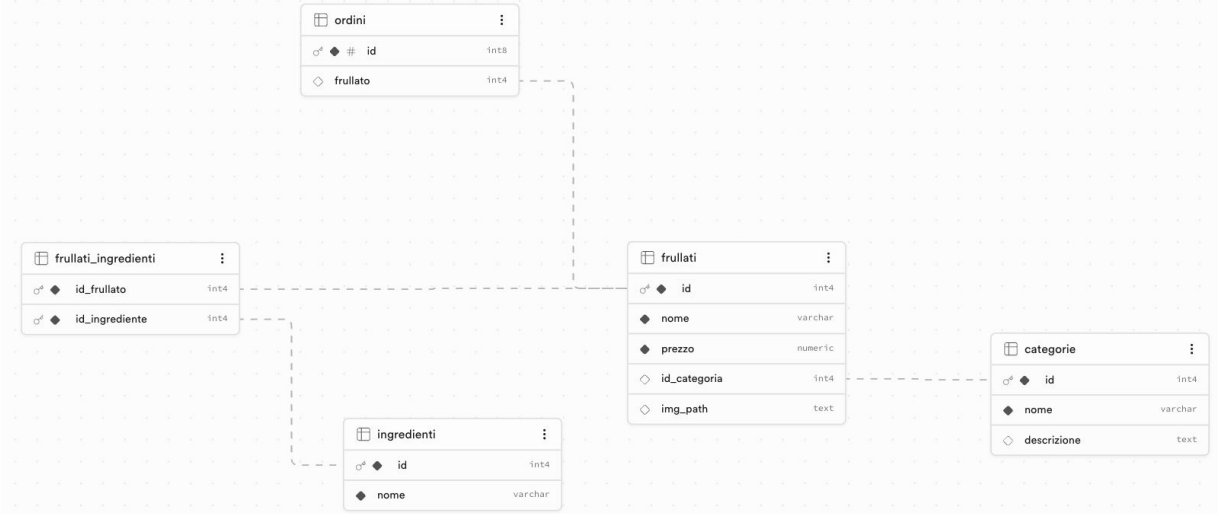
Il database di Sip ‘n Snap

Il database di Sip ‘n Snap implementa il seguente schema ER.



Per l’implementazione viene utilizzato il servizio Supabase, è gratuito e offre un DB relazionale che usa PostgreSQL.

Dallo schema ER precedente ricaviamo il seguente schema logico che è già stato implementato in Supabase.



Le chiamate API

Nell’esempio Sip ‘n Snap sono implementate le seguenti chiamate al server:

- Una chiamata GET per la navigazione in ogni pagina
- Una chiamata GET dalla pagina degli ordini per recuperare dal database tutti i frullati disponibili.
- Una chiamata POST (sempre dalla pagina degli ordini) per eseguire un ordine e chiedere al server di aggiungere una nuova entry nella tabella degli ordini.